

Our CSE-402 Project, Explained From Zero

What we studied, how we did it, what we found —
and how to handle the final defense

CSE-402 Numerical Analysis — Section A

Defense: Week 14, Monday, 2:30 PM

Read this first

This guide assumes you know *nothing* about the project. Every word is explained the first time it appears. Read Parts 1–3 once, slowly, to understand the project. Then use Part 4 to prepare for the defense.

Everyone in the group must be able to explain Part 1 in their own words. The supervisor will ask *each* of you questions, not just one speaker.

Contents

Part 1 — The story in plain words	3
1 The whole project in one paragraph	3
2 How an image AI makes a picture	3
3 Why this is a numerical-analysis problem	4
4 The paper we studied: DPM-Solver	5
Part 2 — How we did it	7
5 Problem: to measure error, you need the right answer	7
6 The three tiers (test problems)	7
7 The three arms (groups of solvers)	7
8 Our measuring tools	8
9 The safety check: Gate G1	8
10 The code, in plain terms	9
Part 3 — What we found	10
11 Finding 1: The theory is right — where its conditions hold	10

12 Finding 2: We reproduced a strange result from the paper, and explained it	10
13 Finding 3: Explosions come from curves, not from stiffness	11
14 Finding 4: The new clock (λ) helps RK4 a lot, on every problem	12
15 Finding 5: Solving the “easy part” exactly is not always a win	12
16 Finding 6 (the headline): Good-looking is not the same as accurate	13
17 What we got wrong at first, and fixed	13
 Part 4 — The defense	 16
18 What the evaluation looks like	16
19 Suggested slides (6 slides, about 8 minutes)	16
20 Live demonstration script	16
21 Who did what (contribution breakdown)	18
22 Question bank	19
23 Words to avoid, and words to use	23
24 Night-before checklist	23
25 Glossary	24
26 One-page number sheet	25

Part 1 — The story in plain words

1 The whole project in one paragraph

AI programs that make pictures (like Stable Diffusion or Midjourney) start from random static and clean it up, step by step, until a picture appears. Each cleaning step needs one run of a big, slow neural network. A research paper called *DPM-Solver* found a clever way to make good pictures in only 10–20 steps instead of 100–250. The paper checked its method by asking “do the pictures look good?” Our project asks a different question, the one a numerical-analysis course cares about: **“Is the method actually solving the math problem accurately?”** We built three test problems, from easy ones where we know the exact answer to a real image AI, and measured accuracy, stability, and cost. Our main discovery: **“looks good” and “is accurate” are not the same thing.** At 10 steps, the method that makes the best-looking pictures is one of the *least* accurate at following the math.

Your 30-second answer to “What is your project?”

“Image-generating AI works by solving a differential equation, one expensive network call per step. The DPM-Solver paper made this fast and judged it by image quality. We judged it the way numerical analysis judges any solver — by accuracy, stability and cost — on three test problems of increasing realism. We found that the theory holds where its assumptions hold, we reproduced a strange result from the paper’s own tables and explained it, and we showed that image quality and mathematical accuracy can rank solvers in exactly opposite order.”

2 How an image AI makes a picture

2.1 Noise

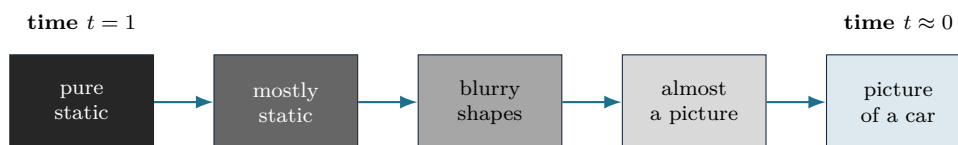
“Noise” means random static, like an old TV with no signal. Every pixel is a random number.

2.2 The trick: learn to remove noise

A **diffusion model** is trained like this:

1. Take a real photo.
2. Add a little noise. Then a bit more. Keep going until it is pure static.
3. Train a neural network to look at a noisy picture and guess *what noise was added*.

Once trained, you can run it *backwards*: start from pure static and keep removing the noise the network predicts. A picture appears.



every arrow = ask the neural network “which noise is here?” once
= **one NFE** (one network call). Network calls are the slow, expensive part.

Word: NFE

NFE = “Number of Function Evaluations” = how many times we run the neural network. It is our unit of **cost**. Fewer NFE = faster picture. Everything in this project is about getting a good answer with few NFE.

3 Why this is a numerical-analysis problem

3.1 What a differential equation is

A **differential equation** (ODE) is a rule that says: “if you are here right now, move in this direction.” The rule for cleaning noise is called the **probability-flow ODE**. The picture x changes with time t according to

$$\frac{dx}{dt} = \underbrace{f(t)x}_{\text{simple part}} + \underbrace{\frac{g^2(t)}{2\sigma(t)}\varepsilon(x,t)}_{\text{neural network part}} .$$

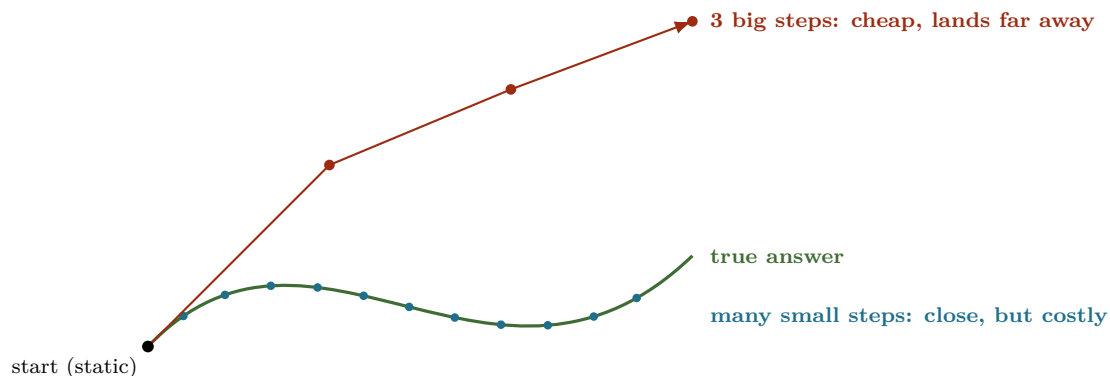
You do not need to memorise this. Just know it has two pieces: a **simple piece** (plain arithmetic) and a **hard piece** $\varepsilon(x,t)$, which is the neural network’s guess of the noise.

3.2 What a solver is

We cannot solve this equation with a pen, so we **walk** along it in steps. At each step we ask “which direction?” and move a bit. The program that decides how to take these steps is a **solver**. The size of one step is called h .

Walking in fog with a compass

You are walking home in thick fog. You have a compass that tells you the direction, but checking it is slow. If you check every 10 metres you get home accurately but slowly. If you check every 500 metres you are fast, but you drift off the road. *Big steps are cheap but inaccurate. Small steps are accurate but expensive.* A smarter walker uses a better rule for each step so that big steps still stay on the road. That smarter rule is what DPM-Solver is.



3.3 The three things numerical analysis measures

A numerical-analysis course judges *any* solver on three things. These are the three questions of our project.

1. **Order (accuracy).** Make the steps half as big. How much smaller does the error get? For an “order p ” method the error shrinks by 2^p :

Method	Order p	Halve the step $\Rightarrow$ error shrinks by	Example
Euler	1	$2\times$	simplest
Midpoint (RK2)	2	$4\times$	
Heun’s (RK3)	3	$8\times$	
RK4	4	$16\times$	classic textbook method

Higher order = error falls faster = better, *if* the theory holds.

2. **Stability.** If steps are too big, some solvers do not just get inaccurate — they *explode*: the numbers grow to huge values or infinity. How big can a step be before that happens?
3. **Cost.** A higher-order method is more accurate per step, but it calls the network more times per step (RK4 = 4 calls per step). After paying for those extra calls, is it still better? We always compare at the same **NFE**, never at the same number of steps.

4 The paper we studied: DPM-Solver

Paper: Lu et al., “*DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps*”, NeurIPS 2022 (a top AI conference). We call it “the base paper.”

4.1 Its two ideas

1. **Solve the easy part exactly.** Remember the equation has a simple part and a hard part. Old solvers approximate both. DPM-Solver solves the simple part *exactly* with a formula, and only approximates the hard (network) part. Less approximating means less error.
2. **Change the clock.** Instead of measuring time with t , measure it with a new variable λ (“lambda”, the half log signal-to-noise ratio). Equal steps in λ automatically become tiny steps near the end of the cleaning, where the picture changes fastest. It is like a walker who slows down on the dangerous part of the road.

It comes in three versions: **DPM-Solver-1, -2, -3** (order 1, 2, 3).

4.2 What it claimed and how it checked

Claim: good pictures in about **10–20 NFE**, instead of 100–250. It checked this with **FID**, a score that measures whether a large set of generated pictures “looks like” real photos (lower FID = better-looking).

4.3 The gap we filled

FID is about *looks*. It does not say whether the solver followed the equation correctly. A picture can look fine even if the solver went to a different place than the true answer. The paper never measured order, stability, or accuracy against a known correct answer. **That missing check is our project.**

Why does the project matter?

“The paper is about a numerical method, but it was only graded like an image generator. We graded it like a numerical method.”

Part 2 — How we did it

5 Problem: to measure error, you need the right answer

Error = (what the solver got) – (the right answer). For a real AI picture, *nobody knows the right answer*. So we built test problems where we *do* know it, from easy to realistic.

Testing a car

First you test a car on a flat track with exact measurements. Then on a winding road. Then in real traffic. A car that works on all three is truly good. One that only works on the track is only good on the track.

6 The three tiers (test problems)

Tier	The problem	How we know the right answer	What it is good for
1	A simple bell-curve (Gaussian) shape of data	An exact formula, pen and paper. Zero reference error.	Clean check of order; test stiffness
2	Data made of 8 points on a curved spiral (“Swiss roll”)	A very accurate solver (SciPy DOP853, tolerance 10^{-13})	Order when the problem is curved (non-linear)
3	A real image AI: <code>google/ddpm-cifar10-32</code> , makes 32×32 colour pictures	A very fine run (201 NFE) of the same network, from the same starting noise	Does it all survive a real network?

Tiers 1 and 2 run on a normal laptop in seconds. Tier 3 needs a graphics card (GPU), so we ran it on **Kaggle**, a free website that lends you a GPU (an NVIDIA T4).

Two words that come up a lot

Stiff: an equation is “stiff” when some parts change very fast and others slowly. Stiff equations make simple solvers explode unless the steps are tiny. Ours gets stiff near the end of cleaning ($t \rightarrow 0$).

κ (“kappa”): in Tier 1, a knob for how stretched the bell-curve is. Bigger κ = stiffer. We tried κ from 1 to 10 000.

7 The three arms (groups of solvers)

DPM-Solver has two ideas mixed together (exact simple part, and the λ clock). To see what *each* idea is worth on its own, we ran three groups:

Arm	What it is	Solvers
A	Old-school solvers, normal clock t	Euler, Midpoint (RK2), Heun (RK3), RK4, AB2
B	The same old-school solvers, but on the new λ clock	the same five
C	DPM-Solver itself (λ clock + exact simple part)	DPM-Solver-1, -2, -3 (and DDIM)

Why three arms?

A vs B = the value of the new clock alone. **B vs C** = the value of solving the simple part exactly, alone.

For arm C we did **not** rewrite DPM-Solver ourselves. We used the authors' own published code (the file `third_party/dpm_solver_pytorch.py`, unchanged) and fed it our exact formulas instead of a network. So we tested the *real* published code.

8 Our measuring tools

- **Error:** the distance between the solver's answer and the right answer (called the L_2 distance — think of it as “how far apart”).
- **Measuring order:** run the solver with many step sizes, plot error against step size on a log-log graph. The slope of the line = the order. A straight line with slope 3 means order 3.
- **Order fitter** (`src/metrics.py`): real curves are only straight in the middle. At very big steps the theory has not “kicked in” yet; at very small steps computer rounding takes over. The fitter finds the straightest middle part automatically.
- **Stability test** (`src/stability.py`): keep making the steps bigger (by halving the number of steps, a method called *bisection*) until the answer explodes. The largest safe step is $h_{\max}$.
- **Crossover:** the point where two solvers' error curves cross — where one stops being better than the other.
- **FID** (Tier 3 only): the image-quality score the paper used.

9 The safety check: Gate G1

Before trusting any number, we checked the code with a test that has only one correct result. The paper proves that **DPM-Solver-1 and an older method called DDIM are the same formula**, just written differently. So two separately written programs must give the same answer, down to computer rounding.

Result: they agree to 5.5×10^{-16} (the limit of computer precision is about 10^{-16}). We passed. If this had failed, nothing else would be worth trusting.

If asked “how do you know your code is correct?”

“Three ways. First, Gate G1: two different formulas the paper proves are equal agree to 10^{-16} . Second, on Tier 1 every solver hits its textbook order, which is impossible if the code were wrong. Third, 129 automatic tests run in under a minute.”

10 The code, in plain terms

Folder / file	What it does
src/schedule.py	The noise “clock”: how much noise at time t ; converts $t \leftrightarrow \lambda$
src/testbeds.py	Tier 1 and Tier 2 test problems and the ODE itself, written once
src/solvers.py	Arm A/B solvers: Euler, Midpoint, Heun3, RK4, AB2, plus <code>integrate()</code>
src/grids.py	Step grids: equal steps in t (arm A) or in λ (arm B)
src/dpm.py	Our own hand-written DPM-Solver-1 and DDIM (for Gate G1)
src/arm_c.py	Connects the authors’ DPM-Solver code to our test problems
src/metrics.py	Error measure and the order fitter
src/stability.py	The stability (explosion) test
src/crossover.py	Finds where two error curves cross
src/tier3.py	Tier 3: the real network and its own noise clock
src/imaging.py	Turns numbers into pictures
tests/	129 automatic checks
notebooks/	13 notebooks (01–12) that run experiments, make tables and plots
results/	All numbers, as spreadsheets (CSV)
figures/	All plots
report/	The formal report and this guide
run_all.sh	One command that reruns every laptop experiment from scratch

Rules we kept the whole way

- **float64 everywhere** on Tiers 1–2 (the most precise normal number type). Less precise numbers would hide the order.
- **Every run has a fixed random seed**, so results repeat exactly.
- **Always compare solvers at equal NFE**, never equal step count.
- Rerunning everything gives the same numbers to 10^{-15} .

Part 3 — What we found

Six findings. For each: the question, what we saw, what it means.

11 Finding 1: The theory is right — where its conditions hold

Question: Does each solver reach its textbook order?

Tiers 1 and 2, at comfortable settings (stopping at $t = 0.2$, many steps): *yes*, every solver, within 0.094 of theory. Euler ≈ 1.01 , RK4 ≈ 3.94 – 4.00 , DPM-Solver-3 ≈ 3.09 .

Tier 3, the real network, at the settings people actually use (stopping at $t = 0.001$, only 10–120 NFE): the measured orders are much lower. RK4 measures 1.21 instead of 4. DPM-Solver-3 measures 2.47 instead of 3.

The important twist (from our control experiment). Is the network to blame? We re-ran Tiers 1–2, which have *no* network and an exact answer, with exactly Tier 3’s settings. They *also* fell short of theory: worst gap 1.91, versus 2.79 on the network. So about 68% of the drop happens with no network at all.

What it means

The order theorem is a promise about *small* steps. At 10–120 NFE the steps are not small enough for the promise to apply yet (this is called **pre-asymptotic**). The network makes it somewhat worse on top. So: the theory is fine; people just use these solvers outside the range where the theory speaks.

Do NOT say this

“The neural network breaks the paper’s smoothness assumption (Assumption B.1), so order collapses.” — We used to think this. It is wrong: the assumption only needs smooth derivatives, and this network uses SiLU activations, which *are* smooth. The control experiment above is the real explanation.

Detail worth knowing: on the network, DPM-Solver-3’s 2.47 becomes **2.995** if you ignore the one measurement that sits on the “noise floor” (the level below which our reference itself is not accurate). So DPM-Solver-3 nearly keeps its order even on the real network.

12 Finding 2: We reproduced a strange result from the paper, and explained it

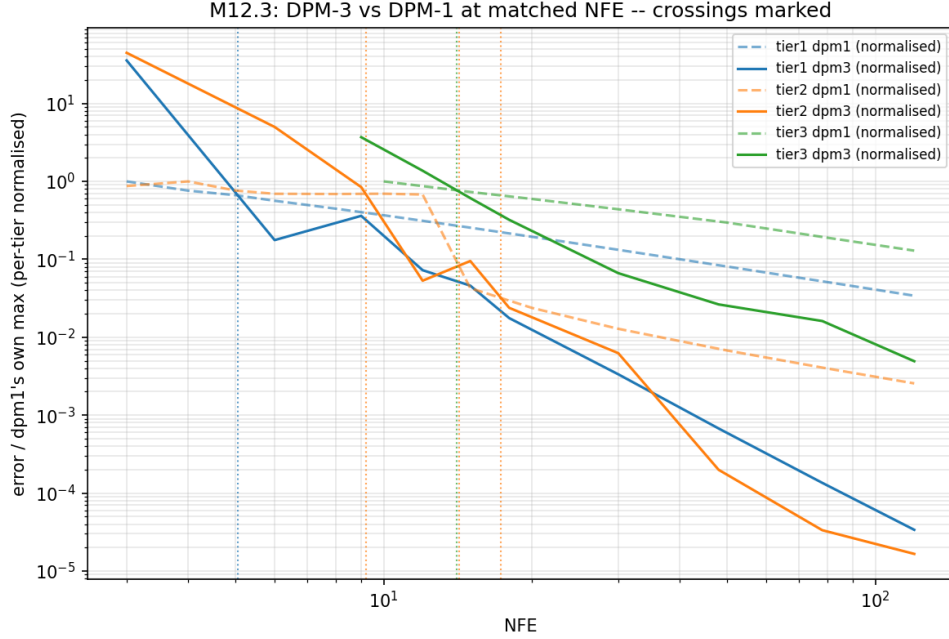
The strange result. In the paper’s own Table 6, at about 10 NFE, DPM-Solver-3 (order 3) is *worse* than DPM-Solver-1 (order 1): FID 24.37 versus 16.69. Higher order losing to lower order. The paper does not explain it.

Why it happens. Order 3 only helps when steps are small. DPM-Solver-3 also spends 3 network calls per step, so at 10 NFE it gets only 3 big steps. Big steps $\Rightarrow$ the order-3 correction makes things worse, not better.

What we measured. Comparing at equal NFE, DPM-Solver-3 starts beating DPM-Solver-1 at about:

Test problem	Crossover NFE
Tier 1 (simple Gaussian)	≈ 5
Tier 2 (curved spiral)	≈ 9 , then crosses again at 14 and 17
Tier 3 (real network, same one as the paper)	≈ 14
Paper Table 6	between 10 and 12

Our real-network number (14) is close to the paper’s (10–12). And the crossover moves later as the problem gets harder.



Solid = DPM-Solver-3, dashed = DPM-Solver-1. Dotted vertical lines mark where they cross. Left of the line, the simpler solver wins.

Say it like this

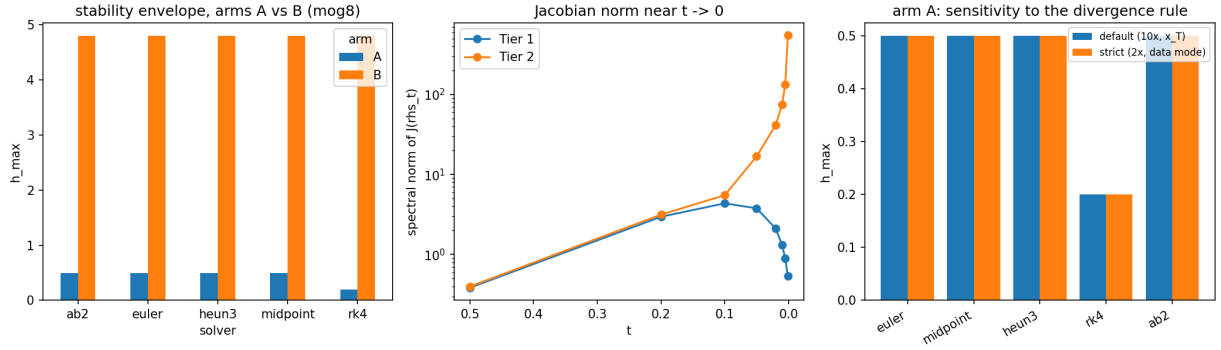
“Higher order is not automatically better. It pays only after a certain budget. On the paper’s own network we measured that budget at about 14 NFE, matching the anomaly in the paper’s Table 6.”

13 Finding 3: Explosions come from curves, not from stiffness

The paper’s claim (section 4.2): old-school solvers can become unstable (explode) with big steps. The paper cites other work but never measures it.

Tier 1 (straight, stiff): no explosion *anywhere*, even at stiffness $\kappa = 10^8$ and with only 2 steps. We found why: near the end the growing stiffness and the shrinking last step cancel out almost exactly, so the danger number stays below 0.9 (Euler needs it below 2 to be safe).

Tier 2 (curved): a real explosion. RK4 in normal time t blows up with fewer than 5 steps: its safe limit is $h_{\max} = 0.1998$. The same RK4 on the λ clock (arm B) never explodes. The reason: near the end, the curved problem is about 1000 $\times$ more “twisty” than the straight one.



Left: largest safe step. Arm A’s RK4 (blue, short) is limited; arm B (orange) is safe everywhere. Middle: how “twisty” the problem gets near the end — Tier 2 (orange) shoots up, Tier 1 (blue) goes down.

What it means

The paper is partly right. Explosions are real, but they come from a *curved* problem, not from stiffness alone. And the λ clock fixes them.

14 Finding 4: The new clock (λ) helps RK4 a lot, on every problem

We compared the same solver on the old clock (arm A) and new clock (arm B), at 120 NFE, running to the very end ($t = 0.001$). Number = how many times smaller the error is on the new clock:

	Euler	Midpoint	RK4
Tier 1	0.69	0.85	48×
Tier 2	9.9	33	8551×
Tier 3 (network)	0.63	0.56	10.6×

For RK4 the new clock is a huge win everywhere. For the simple solvers it is mixed (below 1 means the old clock was actually a little better). Why RK4 gains most: equal λ steps crowd near the end, where RK4 looks hardest.

15 Finding 5: Solving the “easy part” exactly is not always a win

DPM-Solver’s first idea (solve the simple part exactly) helps or hurts depending on the *data*:

- **Concentrated data** (all data near one point, like natural images really are): DPM-Solver wins. At 20 NFE it beats plain Euler at 83% of test points, and DPM-Solver-2 beats RK2 at 100%.
- **Spread-out data** (data as spread as the noise): the old solver’s equation becomes exactly zero, so Euler is *perfect* (error 10^{-17}), while DPM-Solver-1 still has an error (5.9×10^{-3}). Splitting the equation destroyed a lucky cancellation.

This explains why DPM-Solver works so well on real images: image data is concentrated.

16 Finding 6 (the headline): Good-looking is not the same as accurate

This is our most important result.

We generated real pictures on Kaggle with every solver, at 10 and 20 NFE, and scored them two ways:

1. **FID** (the paper’s way): does a set of 5 000 pictures look like real photos?
2. **Accuracy** (our way): how far is each picture from the *correct* picture (the 201-NFE reference from the same starting noise)?

Result: at 10 NFE, the two rankings are exactly opposite. (Rank correlation -1.00 : the best by one measure is the worst by the other.)

Solver, 10 NFE	FID (lower = looks better)	Distance to correct picture
DPM-Solver-2	24.9 (best looking)	180.6 (far)
DPM-Solver-1	44.9	78.4 (close)

DPM-Solver-2 is $2.3\times$ *further* from the correct picture, yet scores $1.8\times$ *better* FID. By 20 NFE the rankings mostly agree again (correlation $+0.60$).

What it means

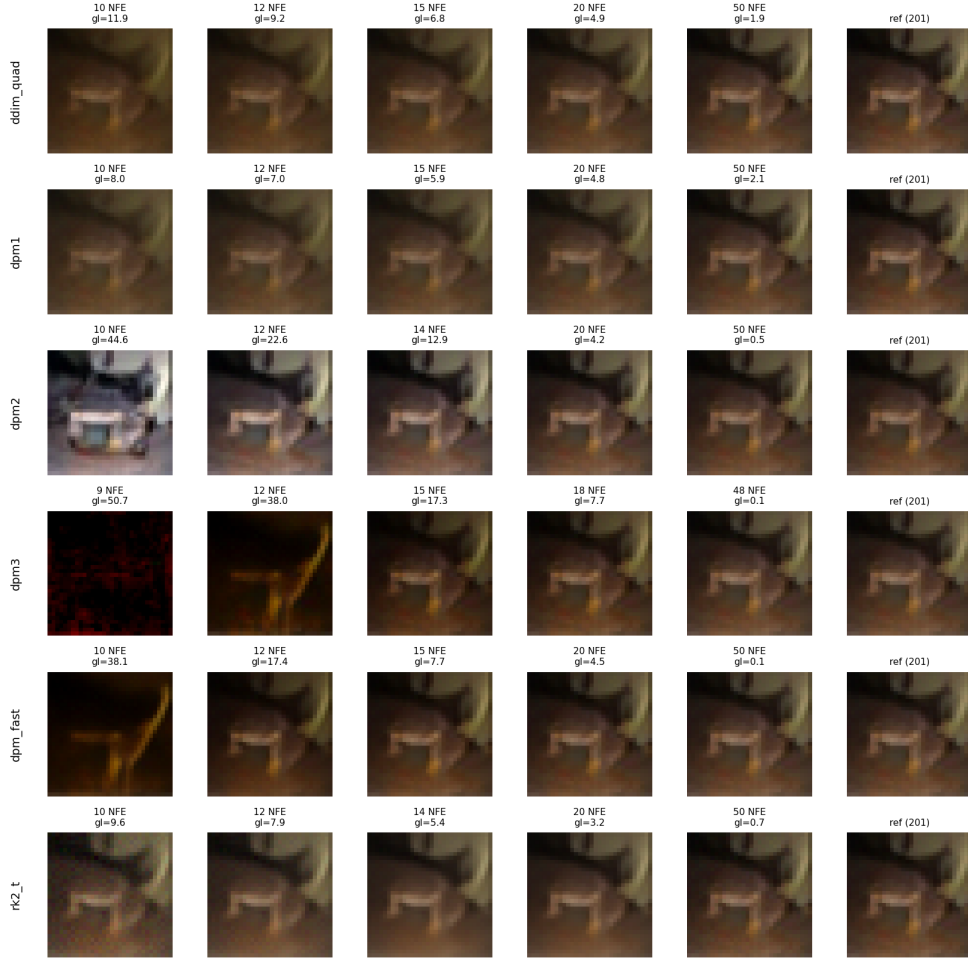
“DPM-Solver makes good images in 10 steps” — **true** (FID agrees). “DPM-Solver solves the equation accurately in 10 steps” — **false**. These are two different claims. The paper proved the first and implied the second. This is exactly why we said solvers must also be judged as numerical methods.

Is our FID trustworthy? Our FID *ranking* matches the paper’s Table 6 (only the top two are swapped). Our absolute FID numbers are about $3\times$ bigger than the paper’s. We checked why (notebook 11b): FID is biased by how many pictures you use. Even *real* photos score about $30\,000/N$ with N pictures — 5.9 at our 5 000, 3.15 at 10 000 (the known correct value, so our FID code works). That explains about half the gap. The other half is a real, unexplained difference in setup. It shifts every solver equally, so **the ranking is still valid**. We compare rankings, not raw FID numbers.

17 What we got wrong at first, and fixed

In the middle of the project a review (24 September) found that some early conclusions compared things under different settings. We fixed them with new experiments (milestones M11–M13). Know this list: a supervisor may have read an older version, and it shows you checked your own work.

Tier 3, google/ddpm-cifar10-32: samples across solvers and NFE (guide's Figure-4 analogue)



Each row is a solver, each column a budget. The last column is the correct picture. “gl” = average difference from the correct picture, in gray levels (0–255): 1 is invisible, 20+ is visibly different. Look at the first column: **DPM-1 (row 2)** gives the *right* picture, a bit blurry (gl 8.0). **DPM-2 (row 3)** gives a *sharp* picture — but a *different* one (gl 44.6).

DPM-3 at 9 NFE (row 4) gives static.

Early claim (wrong)	Corrected claim
The network breaks Assumption B.1, so order collapses	Mostly pre-asymptotic: the same drop happens with no network at the same settings
Order collapses by “a factor of 33”	That compared different settings; matched, it is 1.91 vs 2.79
The stability claim is refuted	Only on the straight problem; on the curved one RK4-in- t really explodes
The λ clock only helps on the real network	It helps RK4 on every tier ($48\times$, $8551\times$, $10.6\times$)
Crossover compared at equal step size	Compared at equal NFE, like the paper: ≈ 14 on the network
FID dropped	FID done at small scale (5k images) as a ranking check

If asked “did anything go wrong?”

“Yes. Our first version compared the analytic tiers and the network under different settings, so it blamed the network for something the settings caused. A review caught it, we re-ran the experiments under matched settings, and the conclusions changed. We kept the corrected version.”

Part 4 — The defense

18 What the evaluation looks like

From the instructor’s post: **Week 14, Monday, 2:30 PM (Section A)**. Your supervisor runs it. It has four parts:

1. **Brief project summary** — base paper, our approach, our extensions and findings.
2. **Live code demonstration** — run code, show outputs (curves, tables).
3. **Results walkthrough** — compare our results to the paper.
4. **Individual Q&A** — the supervisor asks *each* member about code, methods and findings.

Every member must attend and speak. Each must explain their own contribution. Slides are optional but encouraged (4–6 slides).

19 Suggested slides (6 slides, about 8 minutes)

#	Title	What to put	Show
1	The problem	Image AI = solving an ODE; each step costs one network call (NFE). DPM-Solver: 10 steps. Judged only by FID.	fog/steps picture, or the samples grid
2	Our approach	3 tiers $\times$ 3 arms. Measure order, stability, cost. Gate G1 passed (10^{-16}).	the two tables from Part 2
3	Theory holds; practice is pre-asymptotic	Tiers 1–2: order within 0.094. Network at 10–120 NFE: low; mostly the settings, not the network.	<code>figures/12_matched_protocol.png</code>
4	The paper’s Table 6 anomaly	DPM-3 beats DPM-1 only after ≈ 14 NFE on the paper’s network (paper: 10–12).	crossover plot
5	Stability and the λ clock	No explosion on straight Tier 1; real on curved Tier 2; λ fixes it. RK4 gain $48\times$ / $8551\times$ / $10.6\times$.	stability plot
6	<i>Headline</i> : looks $\neq$ accurate	At 10 NFE FID and accuracy rank solvers in opposite order. DPM-2: sharp but wrong.	the samples grid

Tip: Slide 6 is the strongest. If time is short, go $1 \rightarrow 2 \rightarrow 6$.

20 Live demonstration script

Do this *in this order*. Practise it once the day before.

Step 1 — Run all the automatic checks (about 50 seconds)

Open a terminal in the project folder:

```
python -m pytest
```

Expected last line: 129 passed. Say: *“These check every piece: the noise clock, every solver’s order, Gate G1, and the report tables.”*

Step 2 — Show order and Gate G1 live (under 1 second)

Open any notebook in notebooks/ and paste this into a new cell. It uses only our own code, on Tier 1, and runs instantly:

```
import sys, pathlib
p = pathlib.Path.cwd()
while not (p / "pyproject.toml").exists() and p != p.parent:
    p = p.parent
sys.path.insert(0, str(p))

import numpy as np
from src.testbeds import GaussianTier1
from src.grids import grid_t
from src.solvers import integrate
from src.arm_c import sample_dpm_solver
from src.dpm import dpm_solver_1, ddim_step, march
from src.schedule import lmbda, t_of_lmbda
from src.metrics import l2

tb = GaussianTier1(kappa=10.0, d=3, seed=0)      # Tier 1: exact answer known
xT = tb.x_T(5)                                  # 5 starting noises
T, T_END = 1.0, 0.2
exact = tb.exact(xT, T, T_END)                  # the right answer, by formula

print(f"{'solver':<22}{'n=10':>11}{'n=20':>11}{'n=40':>11}  ratio -> order")
cases = [
    ("Euler    (arm A, p=1)", lambda n: integrate(tb.rhs_t, xT, grid_t(T, T_END, n), "euler")[0]),
    ("RK4      (arm A, p=4)", lambda n: integrate(tb.rhs_t, xT, grid_t(T, T_END, n), "rk4")[0]),
    ("DPM-1    (arm C, p=1)", lambda n: sample_dpm_solver(tb.eps, xT, T, T_END, 1, n)[0]),
    ("DPM-3    (arm C, p=3)", lambda n: sample_dpm_solver(tb.eps, xT, T, T_END, 3, 3 * n)[0]),
]
for name, run in cases:
    e = [l2(run(n), exact) for n in (10, 20, 40)]
    r = e[1] / e[2]
    print(f"{'name':<22}{e[0]:>11.2e}{e[1]:>11.2e}{e[2]:>11.2e}  {r:5.1f}x -> {np.log2(r):.2f}")

tg = t_of_lmbda(np.linspace(lmbda(1.0), lmbda(1e-3), 21))
gap = np.max(np.abs(march(dpm_solver_1, tb.eps, xT, tg) - march(ddim_step, tb.eps, xT, tg)))
print(f"\nGate G1: |DPM-Solver-1 - DDIM| = {gap:.1e}  (must be < 1e-14)")
```

Expected output (checked on our machine):

solver		n=10	n=20	n=40	ratio -> order
Euler	(arm A, p=1)	1.89e-01	9.12e-02	4.48e-02	2.0x -> 1.03
RK4	(arm A, p=4)	1.66e-05	1.39e-06	9.65e-08	14.4x -> 3.85
DPM-1	(arm C, p=1)	1.49e-01	7.61e-02	3.84e-02	2.0x -> 0.99
DPM-3	(arm C, p=3)	7.21e-04	8.10e-05	9.58e-06	8.5x -> 3.08

Gate G1: |DPM-Solver-1 - DDIM| = 6.7e-16 (must be < 1e-14)

How to explain it: *“Doubling the steps cuts Euler’s error by 2, so order 1. RK4’s by about 16, so*

order 4. DPM-Solver-3's by 8, so order 3. Exactly the textbook. And Gate G1 shows two different formulas giving the same answer to 10^{-16} ."

Step 3 — Show a full experiment notebook (1–2 minutes)

Open `notebooks/12_controls.ipynb` or `08_crossover.ipynb` and press *Run All*. They run on the laptop. Scroll to the plots while it runs. Each notebook ends with a cell that prints `PASS`.

Step 4 — Tier 3 (do NOT run live)

Tier 3 needs a GPU and 20–40 minutes. Open these two notebooks *with their saved outputs*:

- `notebooks/09_tier3_cifar10.ipynb`
- `notebooks/11_samples_fid.ipynb`

and show the samples grid. Say: “*This ran on a Kaggle T4 GPU; here are the saved outputs.*”

Backup plan if something breaks

- Keep `report/main.pdf` and this guide open in a PDF viewer.
- Keep `figures/` open in a file browser — every plot is there.
- If Python fails, show the notebooks' saved outputs; they already contain every plot and table.
- Never try to fix a bug in front of the supervisor. Say “here is the saved run” and move on.

21 Who did what (contribution breakdown)

Each of you must explain *your own* part. This table is taken from the git commit history — **fix it if your team split the work differently**.

Member	Milestones (from git)	Must be able to explain
Mehemud Azad	M0 project setup and results format; M1 noise clock; M2 Tier-1 test problem; project instructions; the M11–M13 plan after the review; M13 final report; FID floor check (11b)	how λ relates to t ; the Tier-1 exact formula; why FID numbers differ from the paper’s
Sayjad	M3 classic solvers + λ grids; M4 DPM-Solver-1/DDIM + Gate G1; M11 generated pictures + FID; M12 control experiments	how RK4 and AB2 work; Gate G1; the L2-vs-FID result; the four controls
BALLISTIC-robin	M5 order fitter + Tier-1 order experiment; M6 stability test	how the slope is measured; why the fitter uses a window; bisection for $h_{\max}$; why Tier 1 never explodes
Khalid Hasan Tuhin	M7 Tier-2 curved problem + DOP853 reference; M8 crossover study	why Tier 2 needs DOP853; the log-sum-exp trick; the crossover and Table 6
Gourove	M9 Tier-3 engine on Kaggle (real network, its own noise clock); M10 report tables, ledger, <code>run_all.sh</code> ; first LaTeX report; floor-aware order fitting	why Tier 3 needed its own schedule; the noise floor; reproducibility

How to describe your contribution

Use this shape: “*I built [thing]. It does [job]. The key difficulty was [problem], and I solved it by [fix]. You can see it in [file / figure].*” Example: “I built the Tier-3 engine. It runs the real CIFAR-10 network through all three solver groups. The difficulty was that the network uses a slightly different noise schedule than our formulas, about 5% off, so I read the schedule straight from the network’s own settings. It is in `src/tier3.py`.”

22 Question bank

Answers are short on purpose. Say them in your own words.

22.1 Big-picture questions (anyone can be asked)

Q: What problem does your project solve?

The base paper made image-AI sampling fast but only checked picture quality. We checked it as a numerical method: accuracy (order), stability, and cost per network call.

Q: What is the base paper’s main idea?

Two ideas: solve the simple (linear) part of the equation exactly, and measure time with λ instead of t . This gives good images in 10–20 steps instead of 100+.

Q: What is new in your work — what did you add?

The three-arm design that separates the paper’s two ideas; three test tiers with known answers; the first measurements of order and stability; the explanation of the Table 6 anomaly; and showing that FID and accuracy can disagree.

Q: What is your main finding?

At 10 NFE, image quality and mathematical accuracy rank solvers in exactly opposite order. DPM-Solver-2 makes the best-looking pictures but is $2.3\times$ further from the correct one than DPM-Solver-1.

Q: Does your work prove the paper wrong?

No. The paper is right about image quality — our FID ranking matches its Table 6. We add that “good images” is a different claim from “accurate solution.” We also explained one of its unexplained results.

22.2 Numerical-methods questions

Q: What does “order p ” mean?

If you halve the step size, error shrinks by 2^p . Order 1: $2\times$; order 4: $16\times$.

Q: How did you measure the order?

Ran each solver at many step sizes, plotted $\log(\text{error})$ against $\log(\text{step size})$, and took the slope of the straight middle part. Slope = order.

Q: Why is the curve not straight everywhere?

At big steps the theory does not apply yet (pre-asymptotic). At tiny steps computer rounding errors take over. We fit only the straight middle part.

Q: What is NFE and why compare at equal NFE?

Number of network calls — the real cost. RK4 uses 4 per step, Euler 1. Comparing at equal step count would make RK4 look free. So we always compare at equal NFE.

Q: What is a stiff equation?

One where some parts change very fast and others slowly, so simple solvers need tiny steps or they explode. Ours becomes stiff near $t \rightarrow 0$.

Q: What is RK4? What is Euler?

Euler: take one direction reading, step straight. RK4: take four readings inside the step and average them cleverly; much more accurate per step, but 4 network calls.

Q: What is AB2?

Adams–Bashforth 2, a “multistep” method: it reuses the direction from the previous step, so it costs only 1 call per step and still reaches order 2. It needs one extra step to start.

Q: What is DOP853 and why use it?

A very high-accuracy (order 8) solver from SciPy. Tier 2 has no exact formula, so we used DOP853 at extreme accuracy (10^{-13}) as the “right answer,” and checked it by rerunning it at another accuracy.

Q: What is λ ?

Half the log of the signal-to-noise ratio: $\lambda = \log \alpha - \log \sigma$. It gets bigger as the picture gets cleaner. Equal steps in λ put more steps near the end, where things change fast.

Q: Why use `expm1` in DPM-Solver-1?

The formula needs $e^h - 1$. For small h , computing e^h then subtracting 1 loses most digits. The function `expm1(h)` computes it accurately. Small h is exactly where we measure.

Q: What is Gate G1?

A correctness check: the paper proves DDIM and DPM-Solver-1 are the same formula. Our two separate implementations agree to 5.5×10^{-16} , the limit of computer precision.

22.3 Code and implementation questions

Q: Did you write DPM-Solver yourselves?

DPM-Solver-1 yes (it is 4 lines, used for Gate G1). For orders 2 and 3 we used the authors' own published code unchanged and plugged in our exact formula instead of a network. So we tested the real code.

Q: How do you get an “exact answer” for Tier 1?

For bell-curve data the equation splits into independent one-number equations, each with a closed-form solution: $x(t) = x(T)\sqrt{v(t)/v(T)}$. No approximation.

Q: Why float64?

float32 has only about 7 digits; its rounding error (10^{-7}) would flatten our error curves and hide the order.

Q: How do you make sure results are reproducible?

Fixed random seeds, `run_all.sh` reruns everything, and the rerun matches the saved results to 10^{-15} . Plus 129 automatic tests.

Q: Why Kaggle?

Tier 3 runs a real neural network 5 000+ times; that needs a GPU. Our laptops have none. Kaggle gives a free NVIDIA T4 GPU.

Q: What was hard about Tier 3?

The network's noise schedule is a slightly different (step-by-step) version of our formula, about 5% off. Mixing them would add fake error. So Tier 3 reads every schedule value from the network's own settings, for all three arms.

22.4 Results questions

Q: Why is order lower on the real network?

Mostly because 10–120 NFE is not enough steps for the theory to apply yet. We proved this by running the exact problems at the same settings: they also fell short (worst gap 1.91 vs 2.79). The network adds some extra drop on top.

Q: Explain the Table 6 anomaly.

At 10 NFE, DPM-Solver-3 gets only 3 big steps, and an order-3 correction with big steps makes things worse. It only wins after about 14 NFE on the paper’s network; the paper’s own numbers put it at 10–12.

Q: Did you find instability?

Not on the straight Tier 1, even at stiffness 10^8 — we found an exact cancellation that prevents it. On the curved Tier 2, yes: RK4 in normal time explodes below 5 steps. The λ clock fixes it.

Q: Why are your FID numbers bigger than the paper’s?

We used 5 000 pictures, the paper used 50 000, and FID gets worse with fewer pictures. We measured this: about half of the gap. The rest is some setup difference. It shifts all solvers equally, so rankings are still valid, and we only compare rankings.

22.5 Hard or trick questions

Q: Isn’t the order collapse just because neural networks are not smooth?

No. This network uses SiLU activations, which are smooth. And the same drop appears with no network, at the same settings. So it is the budget, not the network.

Q: If DPM-Solver-2 is inaccurate, why does it look better?

It lands on a *sharp* picture that is a real-looking image, just not the one the exact equation leads to. FID only checks “does it look real,” not “is it the right one.” DPM-Solver-1 lands close to the right picture but blurry.

Q: So which solver should people use?

Depends on the goal. For good-looking pictures fast: DPM-Solver-2 or DPM-Solver-fast, as the paper says. For an accurate solution of the equation: the budget matters, and higher order only pays beyond about 14 NFE.

Q: What are the limitations of your work?

One network (CIFAR-10, small pictures); FID with only 5 000 samples, so we compare rankings, not raw values; the Tier-3 reference is itself a numerical run, so errors below its noise floor mean nothing; half of the FID gap is unexplained.

Q: What would you do next?

Test bigger networks and higher resolutions; use 50 000 FID samples; find the source of the remaining FID gap.

Q: Why did you drop the RK45 adaptive solver from the proposal?

An adaptive solver chooses its own step size, so “error vs step size” and “largest safe step” are not defined for it. We added AB2 instead as the multistep method.

23 Words to avoid, and words to use

Avoid	Say instead
“The network violates Assumption B.1”	“Most of the drop is pre-asymptotic; the network adds some”
“We proved the paper wrong”	“We tested it as a numerical method and added findings”
“DPM-Solver is inaccurate”	“At 10 NFE no solver is accurate; DPM-Solver trades accuracy for sharp images”
“Order 3 is always better”	“Order 3 wins only above a budget, ≈ 14 NFE here”
“Our FID is 24.9” (as an absolute claim)	“Our FID ranking matches the paper’s”
“Stability was refuted”	“No instability on the linear problem; real on the curved one”

24 Night-before checklist

- ☐ `git pull` so everyone has the latest version.
- ☐ Python environment works: `python -m pytest` prints 129 passed.
- ☐ Jupyter opens: `jupyter lab`. The live-demo cell runs.
- ☐ Notebooks 09 and 11 open *with outputs visible* (no GPU needed to view them).
- ☐ `report/main.pdf`, this guide, and `figures/` open in tabs.
- ☐ Laptop charged; charger packed. No internet needed for the demo.
- ☐ Each member has practised their 1-minute contribution speech.
- ☐ Each member can give the 30-second project summary (Section 1).
- ☐ Everyone has read the “Do NOT say” list above.

25 Glossary

Word	Meaning
Diffusion model	Image AI that learns to remove noise; makes pictures by starting from static
Noise	Random static
ODE	A rule saying which direction to move from where you are
Probability-flow ODE	The specific ODE that turns static into a picture
Solver	A method that follows an ODE in steps
Step size h	How far one step goes
NFE	Number of network calls = cost
Order	How fast error shrinks when steps get smaller
Pre-asymptotic	Steps still too big for the order theory to apply
Stable / unstable	Stays bounded / explodes to huge numbers
Stiff	Parts change at very different speeds; hard for simple solvers
κ (kappa)	Stiffness knob in Tier 1
λ (lambda)	The “new clock” DPM-Solver uses (half log signal-to-noise)
t_{end}	Where we stop: 0.001 means almost fully clean
Arm A / B / C	Old solvers on t / old solvers on λ / DPM-Solver
Tier 1 / 2 / 3	Bell curve / curved spiral / real image network
Euler, RK2, RK3, RK4	Classic solvers of order 1, 2, 3, 4
AB2	A method that reuses the last step’s direction
DDIM	An older sampler; mathematically equal to DPM-Solver-1
DPM-Solver-fast	The paper’s recommended mix of orders for small budgets
DOP853	A very accurate order-8 solver used as “truth” on Tier 2
L_2 error	Distance between two answers
FID	Image-quality score: do generated pictures look like real ones (lower = better)
Gate G1	Our code check: DDIM and DPM-Solver-1 must agree to 10^{-16}
Crossover	Where two solvers’ error curves cross
Noise floor	The level of error below which our reference itself is not accurate
Kaggle	Free website that lends GPUs
GPU	Graphics card; runs neural networks fast
Seed	A fixed starting number for randomness, so runs repeat exactly

26 One-page number sheet

Fact	Number
Gate G1: DDIM vs DPM-Solver-1	agree to 5.5×10^{-16}
Tiers 1–2 order, comfortable settings	within 0.094 of theory
Tier 3 (network) worst gap from theory	2.79 (RK4: 1.21 instead of 4)
Tier 2 at Tier 3’s settings, no network	worst gap 1.91 (68% of the network’s)
DPM-Solver-3 on network, floor excluded	2.995 (theory 3)
Crossover DPM-3 vs DPM-1 at equal NFE	T1 ≈ 5 , T2 ≈ 9 , T3 ≈ 14 ; paper 10–12
Tier 2 RK4-in- t safe step	$h_{\max} = 0.1998$ (explodes below 5 steps)
λ -clock gain for RK4	$48\times$, $8551\times$, $10.6\times$
FID at 10 NFE (ours, 5k)	DPM-2 24.9, DPM-fast 31.3, DDIM 39.5, DPM-1 44.9, DPM-3 143
FID at 10 NFE (paper, 50k)	DPM-fast 6.42, DPM-2 7.90, DDIM 13.58, DPM-1 16.69, DPM-3 24.37
FID vs accuracy rank correlation	−1.00 at 10 NFE, +0.60 at 20 NFE
DPM-2 vs DPM-1 at 10 NFE	$2.3\times$ further, $1.8\times$ better FID
FID floor of real photos	$\approx 30\,000/N$: 5.9 at 5k, 3.15 at 10k
Discrete vs continuous schedule	about 5% difference in f
Automatic tests	129, run in under a minute

Every number in this guide comes from the files in `results/` and matches `report/main.pdf` and `docs/PROJECT_EXPLANATION.md`. Base paper: C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, J. Zhu, *DPM-Solver*, NeurIPS 2022, arXiv:2206.00927.